

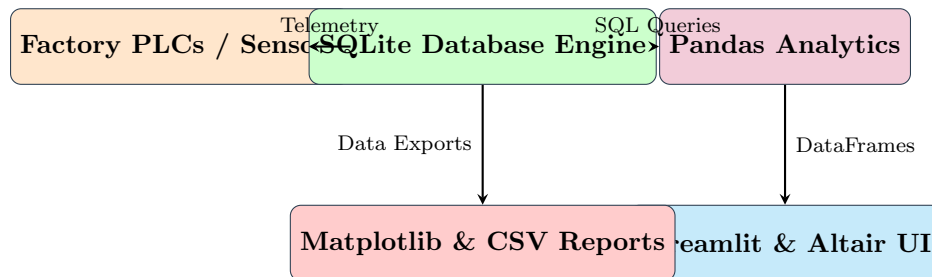
# Industrial SCADA & Telemetry Systems

## Software Engineer Intern & Junior Developer Handbook

---

An End-to-End Practical Guide to Architecture, SQLite Relational Modeling, and Real-Time Dashboards for Plush Toy Manufacturing

---



**Author:** EnigmaK9 Engineering Team  
**Repository:** [israel-pineapple](#)  
**Target Audience:** Junior Software Engineers & SWE Interns

**Version:** 1.0.0  
**Platform:** Linux / macOS / Windows  
**Language:** Python 3.10+

## Contents

---

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction: The Big Picture (What is SCADA?)</b>              | <b>2</b> |
| 1.1      | Physical Floor vs. Software Layer . . . . .                        | 2        |
| 1.2      | Why This Project Matters . . . . .                                 | 2        |
| <b>2</b> | <b>System Architecture &amp; Component Diagram</b>                 | <b>2</b> |
| <b>3</b> | <b>Database Engineering with SQLite</b>                            | <b>2</b> |
| 3.1      | Database Schema . . . . .                                          | 3        |
| 3.2      | Industrial KPI Mathematical Formulas . . . . .                     | 3        |
| <b>4</b> | <b>Code Walkthrough: File by File</b>                              | <b>3</b> |
| 4.1      | db_setup.py (Database Initializer & Telemetry Simulator) . . . . . | 3        |
| 4.2      | app_dashboard.py (Streamlit Real-Time Dashboard) . . . . .         | 3        |
| 4.3      | dashboard_peluches.py (Matplotlib Shift Reporter) . . . . .        | 4        |
| 4.4      | dashboard_web.html (Zero-Dependency Web Client) . . . . .          | 4        |
| 4.5      | generar_csv.py (ERP / Spreadsheet Exporter) . . . . .              | 4        |
| <b>5</b> | <b>Practical Execution: Step-by-Step Guide</b>                     | <b>4</b> |
| <b>6</b> | <b>SWE Intern Field Rules &amp; Best Practices</b>                 | <b>5</b> |

# 1 Introduction: The Big Picture (What is SCADA?)

---

Welcome to industrial software engineering! As a software engineer working on modern manufacturing systems, you bridge the gap between physical machinery and cloud/web applications.

## 1.1 Physical Floor vs. Software Layer

In a manufacturing plant:

- **Machines & Stations:** Physical lines perform mechanical tasks (cutting fabric, stuffing plush bodies with synthetic fiber, stitching embroidered faces).
- **PLCs (Programmable Logic Controllers):** Small, rugged computers that read sensors (counting optical pulses as completed toys pass a laser, monitoring thermocouples on hot needles).
- **SCADA (Supervisory Control and Data Acquisition):** The supervisory system that gathers telemetry pulses, saves them to a database, calculates KPIs, and visualizes them on screens for operators and plant managers.

## 1.2 Why This Project Matters

Without a centralized telemetry platform, if Station 1 starts overheating and ruining teddy bears at 12:00 PM, nobody notices until hundreds of broken bears pile up in the trash bin. With our platform, the system flags a warning the second the scrap rate crosses 5% or head temperature exceeds safe operating thresholds.

# 2 System Architecture & Component Diagram

---

The repository `israel-pineapple` is organized into four clean layers following the **Separation of Concerns (SoC)** software design principle:

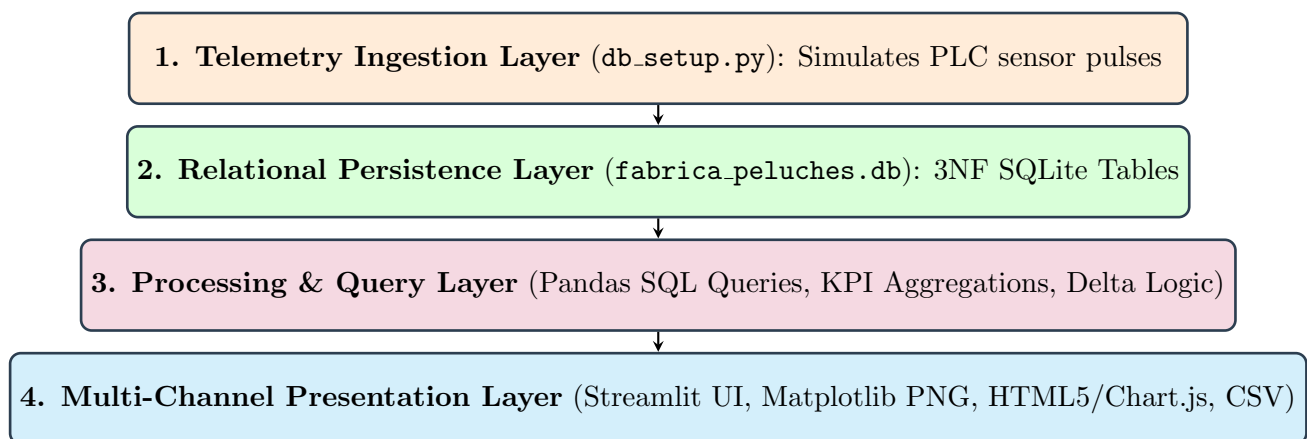


Figure 1: Four-Layer Architecture of the SCADA Monitoring System.

# 3 Database Engineering with SQLite

---

Rather than storing unstructured data, we use a structured relational database with SQLite. It requires zero server setup and stores everything inside a single, fast local file.

### 3.1 Database Schema

The database `fabrica_peluches.db` models the factory with two relational tables:

| Table            | Column              | Data Type           | Description                              |
|------------------|---------------------|---------------------|------------------------------------------|
| lineas           | id_linea            | INTEGER PRIMARY KEY | Unique line identifier                   |
|                  | nombre              | TEXT NOT NULL       | Name of station (e.g., Sewing Line 1)    |
|                  | modelo_peluche      | TEXT NOT NULL       | Product model (e.g., Classic Teddy Bear) |
| produccion_scada | id                  | INTEGER PRIMARY KEY | Auto-incrementing record ID              |
|                  | id_linea            | INTEGER (FK)        | References <code>lineas(id_linea)</code> |
|                  | fecha_hora          | TEXT NOT NULL       | Shift hour (e.g., 08:00, 09:00)          |
|                  | piezas_buenas       | INTEGER NOT NULL    | Count of approved toys produced          |
|                  | piezas_defectuosas  | INTEGER NOT NULL    | Count of defective scrap toys            |
|                  | temperatura_maquina | REAL NOT NULL       | Tool head temperature in Celsius (°C)    |

Table 1: Database Schema Definition.

### 3.2 Industrial KPI Mathematical Formulas

When pulling data from SQLite into Python using Pandas, the business logic evaluates key production indicators:

$$\text{Total Good Units} = \sum_{i=1}^n \text{piezas\_buenas}_i \quad (1)$$

$$\text{Total Scrap Units} = \sum_{i=1}^n \text{piezas\_defectuosas}_i \quad (2)$$

$$\text{Scrap Rate (\%)} = \left( \frac{\text{Total Scrap}}{\text{Total Good} + \text{Total Scrap}} \right) \times 100 \quad (3)$$

$$\text{Average Temperature} = \frac{1}{n} \sum_{i=1}^n \text{temperatura\_maquina}_i \quad (4)$$

## 4 Code Walkthrough: File by File

### 4.1 db\_setup.py (Database Initializer & Telemetry Simulator)

This script builds the database schema and generates historical shift records. It purposefully creates an equipment malfunction in Line 1 at 12:00 to verify alert triggers:

```
# Deliberate machine anomaly simulation
if hora_str == "12:00":
    buenas = 25
    malas = 10      # Scrap spike!
    temp = 85.5     # Overheating needle!
```

### 4.2 app\_dashboard.py (Streamlit Real-Time Dashboard)

This is the core interactive web app. Key features:

1. **Dynamic Line Filtering:** The sidebar lets the operator pick a production line. Under the hood, it executes a parameterized SQL query:

```
SELECT p.fecha_hora, p.piezas_buenas, p.piezas_defectuosas,
       p.temperatura_maquina, l.nombre, l.modelo_peluche
FROM produccion_scada p
JOIN lineas l ON p.id_linea = l.id_linea
WHERE p.id_linea = ? ORDER BY p.id ASC;
```

2. **Live SCADA Pulse Button:** Clicking the sidebar button executes an instant SQL INSERT and re-renders all Altair charts on the fly.

### 4.3 dashboard\_peluches.py (Matplotlib Shift Reporter)

A batch utility designed for cron jobs or automated email attachments. It connects to SQLite, renders a dual-axis trend graph with custom styling, and saves `dashboard_peluches.png`.

### 4.4 dashboard\_web.html (Zero-Dependency Web Client)

A self-contained HTML5 file utilizing **Chart.js** via CDN. It demonstrates how factory displays on the shop floor can render telemetry charts without requiring a Python environment.

### 4.5 generar\_csv.py (ERP / Spreadsheet Exporter)

Exports hourly operational metrics into `scada_peluches.csv` formatted with embedded spreadsheet formulas for automated defect percentage calculations.

## 5 Practical Execution: Step-by-Step Guide

### 1. Clone the Repository:

```
git clone https://github.com/EnigmaK9/israel-pineapple.git
cd israel-pineapple
```

### 2. Initialize Virtual Environment:

```
python3 -m venv .venv
source .venv/bin/activate
```

### 3. Install Dependencies:

```
pip install -r requirements.txt
```

### 4. Seed Database:

```
python db_setup.py
```

### 5. Launch Interactive Dashboard:

```
streamlit run app_dashboard.py
```

## 6 SWE Intern Field Rules & Best Practices

---

- **Never Use String Formatting for SQL:** Always use parameterized tuples (`param,`) to prevent SQL injection.
- **Handle Division by Zero:** If a newly commissioned machine has produced 0 units,  $(0 + 0) = 0$ . Always check `if total > 0` before computing ratios.
- **Idempotent Scripts:** Database setup scripts must be safe to run multiple times without corrupting state or crashing on duplicate keys.
- **Commit Semantics:** Use descriptive conventional commit messages (`feat:`, `fix:`, `docs:`) to maintain a clean git history.